

LLM Appendix

prompt:görev paylaşımı sonucu RagOrchestrator & Pipeline kısmını ben yazacağım ve bunun için tek günüm var. nasıl yazabilirim adım adım bir roadmap çizelim

LLM Answer:

```
> 1.1. PipelineStage Interface
Pipeline'daki bütün aşamalar buna uyacak.

java
public interface PipelineStage {
    void run(PipelineContext context, TraceBus traceBus);
}

> 1.2. PipelineContext class
Tüm pipeline boyunca taşınacak veriler.

java
public class PipelineContext {
    private String question;
    private Intent intent;
    private List<String> terms;
    private List<Hit> hits;
    private Answer answer;

    // getters / setters
}

> 1.3. RagOrchestrator (Template Method uygulanacak ana sınıf)

java
public class RagOrchestrator {
    private List<PipelineStage> pipeline;
    private TraceBus traceBus;

    public RagOrchestrator(List<PipelineStage> pipeline, TraceBus traceBus) {
        this.pipeline = pipeline;
        this.traceBus = traceBus;
    }

    public Answer run(String question) {
        PipelineContext ctx = new PipelineContext();
        ctx.setQuestion(question);

        for (PipelineStage stage : pipeline) {
            stage.run(ctx, traceBus);
        }

        return ctx.getAnswer();
    }
}

> 4.1. ComponentFactory oluştur

java
public class ComponentFactory {
    public static IntentDetector createIntentDetector(Config cfg) { ... }
    public static QueryWriter createQueryWriter(Config cfg) { ... }
    public static Retriever createRetriever(Config cfg) { ... }
    public static Ranker createRanker(Config cfg) { ... }
    public static AnswerAgent createAnswerAgent(Config cfg) { ... }
}

★ BÖLÜM 5 – Pipeline'ı Birleştir (30 dakika)

java
public RagOrchestrator build(Config cfg) {
    IntentDetector id = ComponentFactory.createIntentDetector(cfg);
    QueryWriter qw = ComponentFactory.createQueryWriter(cfg);
    Retriever rr = ComponentFactory.createRetriever(cfg);
    Ranker rr = ComponentFactory.createRanker(cfg);
    AnswerAgent aa = ComponentFactory.createAnswerAgent(cfg);

    List<PipelineStage> stages = List.of(
        new IntentDetectionStage(id),
        new QueryWritingStage(qw),
        new RetrievalStage(rr),
        new RankingStage(rr),
        new AnsweringStage(aa)
    );

    TraceBus bus = new TraceBus();
    bus.addSink(new JsonTraceSink("logs/run" + System.currentTimeMillis() + ".json"));

    return new RagOrchestrator(stages, bus);
}

★ BÖLÜM 6 – CLI Çalıştırma Kodunu Yaz (30 dakika)

java
public class Main {
    public static void main(String[] args) {
        Config cfg = Config.load(args[0]);
        String question = args[1];

        RagOrchestrator orch = OrchestratorBuilder.build(cfg);
        Answer ans = orch.run(question);

        System.out.println(ans.getText());
    }
}

> 3.1. TraceBus
Observer gibi çalışacak.

java
public class TraceBus {
    private List<TraceSink> sinks = new ArrayList<>();

    public void addSink(TraceSink sink) { sinks.add(sink); }

    public void publish(String stage, Object input, Object output, long startTime) {
        TraceEvent ev = new TraceEvent(stage, input, output, System.currentTimeMillis() -
            startTime);
        sinks.forEach(s -> s.record(ev));
    }
}

> 3.2. TraceSink interface

java
public interface TraceSink {
    void record(TraceEvent event);
}

> 3.3. JsonTraceSink

java
public class JsonTraceSink implements TraceSink {
    private BufferedWriter writer;

    public JsonTraceSink(String filePath) throws IOException {
        writer = new BufferedWriter(new FileWriter(filePath, true));
    }

    @Override
    public void record(TraceEvent event) {
        try {
            writer.write(event.toJson());
            writer.newLine();
            writer.flush();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Prompt: I am writing a TextNormalizer for Turkish texts. If I use the standard toLowerCase() method, I might lose information regarding 'I' and 'İ'. What is the safest way to normalize this without external libraries?

AI Answer: Standard Java toLowerCase() is locale-dependent and unsafe for the Turkish 'I' and 'İ' pair. You should sanitize the string manually before tokenizing.

Recommended Flow:

- Explicit Replacement:** First, replace 'İ' with 'i' and 'I' with 'i'.
- Lowercasing:** Now it is safe to apply .toLowerCase().
- Filtering:** Use a whitelist Regex to remove punctuation but keep Turkish characters: `[^a-z0-9ğüşöçı]`.
- Tokenization:** Finally, split by whitespace to get your token list.

```
Java

public String normalize(String text) {
    return text
        .replace("İ", "i") // Handle dotted I
        .replace("I", "i") // Handle dotless I
        .toLowerCase() // Now safe to lowercase
        .replaceAll("[^a-z0-9ğüşöçı]", " ") // Keep only Turkish letters & digits
        .replaceAll("\\s+", " ") // Collapse multiple spaces
        .trim();
}
```

Prompt: In my project, I want to create a RAG system that generates template-based answers by matching questions with the most relevant text snippets from a knowledge base without using LLM. Instead, the system will generate an answer based on the best hits sorted during the retrieval phase. I want to develop the AnswerAgent component. Package: rag.answer, classes: Answer, AnswerAgent, TemplateAnswerAgent, CitationValidator, _register_ First, could you provide me with the outline of the Answer.java code?

AI Answer: I can write an example design for the Answer.java file, which will fit the general RAG architecture and the components you described.

The Answer.java file below is a simple and extensible model, designed to accommodate the fields a typical answer object might need (answer text, sources/citations used, score, etc.).

```
java
package rag.answer;

import java.util.List;

/**
 * Represents the final answer produced by the AnswerAgent.
 * It contains:
 * - the generated answer text
 * - the citations (ids or snippets of retrieved chunks)
 * - an optional confidence score
 */
public class Answer {

    /** The final answer text generated by the agent. */
    private final String text;

    /** The list of citation IDs or text snippets used to construct this answer. */
    private final List<String> citations;

    /** Optional confidence score based on retrieval ranking or heuristics. */
    private final double confidence;
}
```

```
public Answer(String text, List<String> citations, double confidence) {
    this.text = text;
    this.citations = citations;
    this.confidence = confidence;
}

public String getText() {
    return text;
}

public List<String> getCitations() {
    return citations;
}

public double getConfidence() {
    return confidence;
}

@Override
public String toString() {
    return "Answer{" +
        "text='" + text + '\'' +
        ", citations=" + citations +
        ", confidence=" + confidence +
        '}';
}
```